Exploring the ontological digital thread

What a thread is, what it is not, and how relationships compose into a process

Dr. Zachary Welz (Peerless)

2026-07-08

digital thread | ontology | digital engineering | knowledge graphs | provenance | INCOSE DEIX

What a thread is, what it is not, and how individual relationships compose into a connected digital-engineering process. Written alongside the INCOSE DEIX Ontology Working Group's discussions on the digital thread and authoritative source of truth.

The word "thread" is doing two jobs at once in most conversations: the single typed relationship between two artifacts, and the connected web of all such relationships across a lifecycle. Keeping those apart (and keeping the thread apart from the things it connects and the machinery that realizes it) is what lets trust, authority, and coverage claims stay meaningful instead of dissolving into "everything is connected to everything." This note fixes the terms, then maps a working-group traceability example onto them and proposes a cleaner cut.

A note on scope and attribution

The thinking here was shaped by discussions in the **INCOSE DEIX Ontology Working Group** on the digital thread and the authoritative source of truth. I am grateful to that group for the conversations that sharpened these distinctions.

What follows is my own opinion and my own reading of those discussions. It is not a work product of the working group, it has not been reviewed or endorsed by it, and it should not be taken to represent the position of INCOSE, of the DEIX Ontology Working Group, or of any of their members. Where I have taken a side on a contested point, that judgment is mine.

A thread is a typed assertion, not a mechanism

A digital thread, in the atomic sense, is a **persistent, addressable, typed, provenance-bearing assertion that a specific relation holds between two referenced information elements**. It is an edge, not an execution. It references its endpoints by identity; it does not contain them, and it is not the transport that moves data between them.

The ends are **endpoints**: a source (subject) and a target (object). They are separate objects from the thread. The content that sits at an endpoint (a value, a model element) belongs to the node; the thread holds only the two identities plus what it asserts about their relation.

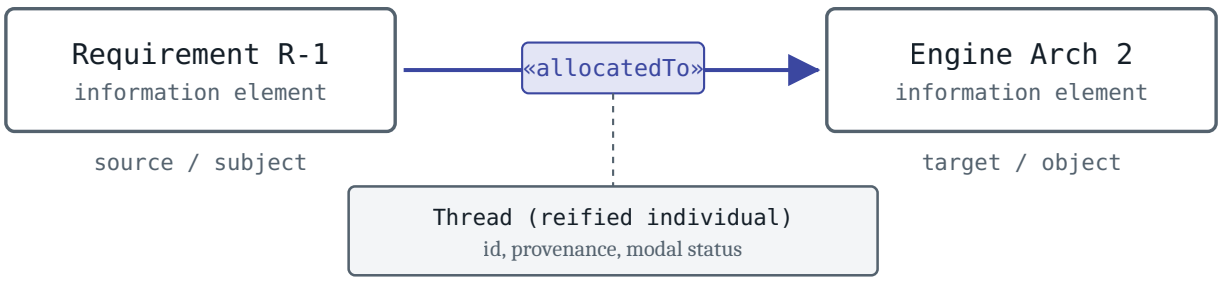

Figure 1. The thread is the typed, addressable edge; the endpoints are the nodes it references, not part of it.

What the thread contains is exactly three things, and nothing else: the *predicate* (which typed relation holds); the two *endpoint references* (identities, not copies); and *provenance and governance annotations on the statement itself* (who asserted it, when, derived from what, at what authority and modal status). The last item is why a thread must be *reifiable* as a first-class individual: you cannot hang trust, time, or authority on a bare triple.

The two primitive kinds differ in *where their content sits*, which is the reason the co-reference / semantic distinction is the primary axis and not static-versus-dynamic:

- Case 1 (co-reference). The thread's entire content is a constraint over the endpoints ("these denote one fact and should hold the same value"). Almost nothing is local; it dereferences to the nodes.
- Case 2 (semantic relation). The content is edge-resident. The predicate (verifies, satisfies, allocates) *is* the information, and it is understandable from the endpoint identities alone.

The triple versus its descriptor set. Boiling the thread down to a typed triple is right as an account of what it *is*, but a triple stripped of context cannot carry intent. Some of what a reader needs is intrinsic and can be read off the edge (the predicate, the endpoint types); the rest (authority, provenance, modality) is a contextual overlay that has to be captured deliberately. To describe a thread so that a person or a tool can act on it, record at least the following around the triple.

Facet	What to record, and why it matters
The triple (what it IS)	(source, predicate, target) with a stable identity. This alone is the minimum that makes it a thread.
Endpoints	each end's identity, type, and modal / configuration status (pinned to a version, or floating).
Predicate	the specific typed relation: Case 1 co-reference, or a named Case 2 sub-relation. Not bare "trace".
Authority	

Facet	What to record, and why it matters
	which endpoint is authoritative for the fact, if any (the ASoT node role, and whether it is granted, realizable, or realized).
Provenance	who asserted the edge, when, from what, by which activity. The evidence behind any trust claim.
Modality	actual, proposed, prescribed, allocated. Whether the endpoints are realized or merely expected.
Cardinality & coverage	one-to-one versus set-to-set, and whether the set is complete. A convergence on a shared endpoint is coverage, not identity.

The first row is the ontology; the rest is what a thread ontology should make first-class rather than optional, because that is where the intent of the digital-engineering process actually lives. A triple you cannot situate (whose ends, authority, and provenance are left unstated) is a fact without a context, and it will be read differently by every tool that touches it.

What is, and what is not, a digital thread

Run each candidate against the essentiality test from the research: a thread **crosses a boundary** (tool, discipline, organization, lifecycle phase), **bears provenance**, is **traversable/queryable**, and (Case 1 only) **references an authoritative value**. Each "no" below is a mechanism whose essence is *doing*; a thread's essence is a persistent relation that survives the doing.

Candidate	Thread?	Why
Foreign key (intra-schema)	no	No boundary crossed, no provenance, not a governed assertion. It is a pointer inside one store.
Pipeline ETL / CI-CD / notebook	no	An *activity*. On execution it *generates* threads (lineage). It is a thread producer, not a thread. wasGeneratedBy, not the relation itself.
Digital workflow read / summarize / respond	no	An *orchestration* (control flow plus intent). It consumes and produces threads. Reorder its steps and it changes; the artifact relations need not.
API OpenAPI / GraphQL / SPARQL	no	An *access interface* and transport. You *walk* threads through it. OSLC is the tell: the API implements threads, but the thread is the RDF assertion it lets you retrieve.
OSLC typed link between two resource URIs	yes	The paradigm: directional, single-owned, provenance-bearing, traversable, crossing tool boundaries.
Requirement satisfiedBy Verification (reified)	yes	A typed Case 2 relation with an identity and provenance; the canonical engineering thread.

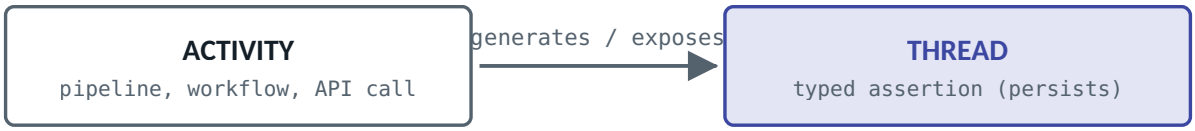

Figure 2. The job runs and ends; the assertion remains. Confusing the two is a cousin of mapping a case onto a technology.

From threads to *the* digital thread

The singular "the digital thread" (the connected digital-engineering process) is not one edge; it is a **connected, traversable graph of many typed edges** spanning tool, discipline, and lifecycle boundaries. The atomic thread is the relationship; the canonical digital thread is the *traversal closure* over the set of relationships. There is no crisp standard name for the aggregate, which is a large part of why it gets conflated with the edge.

Terminology worth adopting: reserve thread (or thread edge) for the atomic typed relation, and use digital thread (aggregate) or thread graph for the connected whole. End-to-end composite relations ("this requirement is evidenced by that test result") are named with *property chains*, not stored as extra edges.

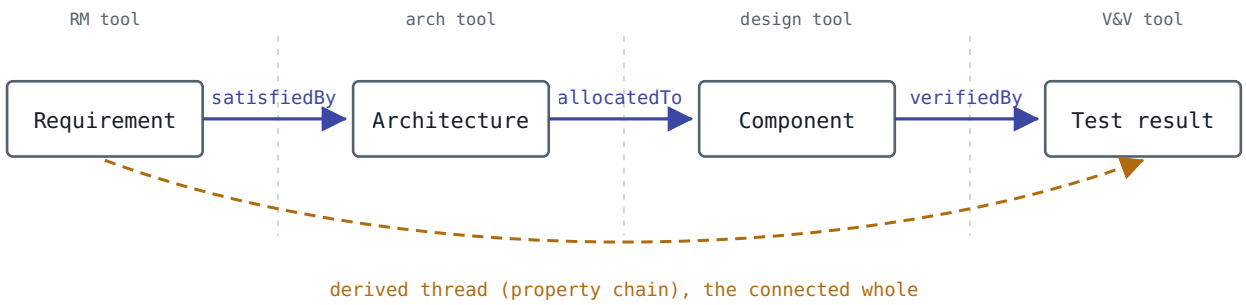

Figure 3. Atomic edges crossing boundaries compose into a traversable graph; the end-to-end relation is derived, not separately stored. A pile of edges that does not connect is not a digital thread.

A working-group traceability example, mapped

The example below was offered as a data-traceability model: three engine requirements, an intermediate "arch trace" per requirement, two architecture elements (with requirement 3 pointing intentionally at the same target as requirement 2), and a bag of process metadata that flows into an ASoT evaluation which can emit new process metadata.

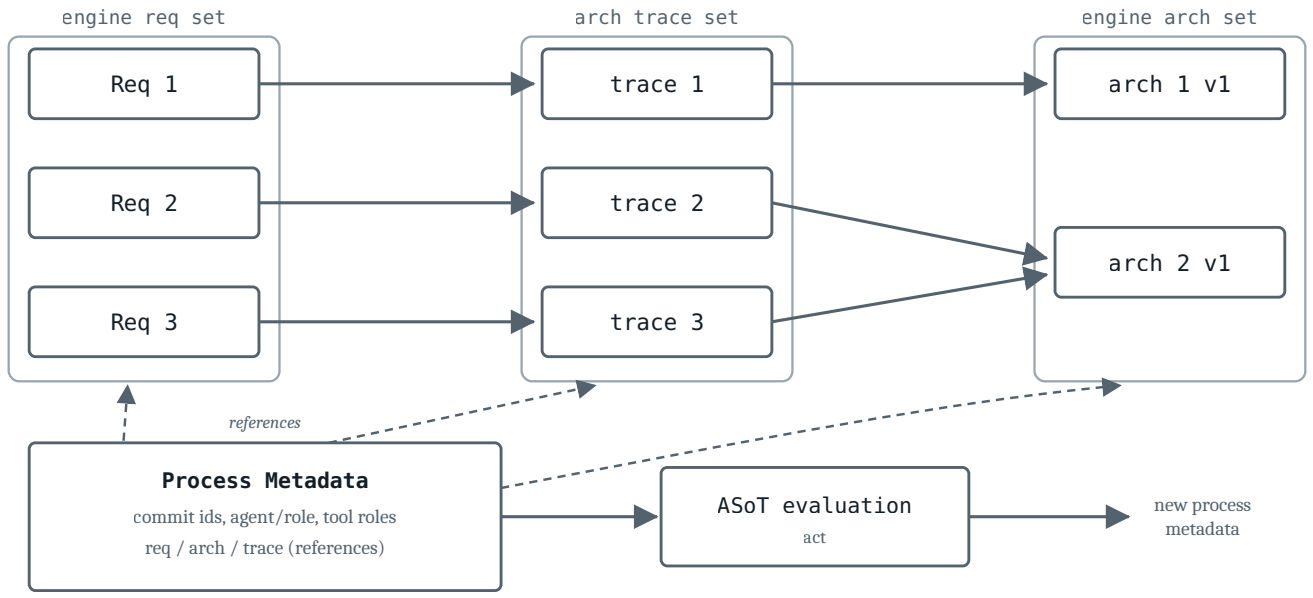

Figure 4. The model as provided. Every link is the same undifferentiated color, which is exactly the ambiguity to resolve.

Read against the concepts, the model gets one important thing right and blurs four others.

- **Right instinct (keep it):** the arch trace id is a *reified Case 2 edge*. Giving the relationship an identity so other facts can attach to it is precisely correct. Naming it for what it is (a SemanticRelationThread individual with a source, a target, and a specific predicate) removes the puzzle of a third "artifact" that is really an edge.
- **Under-typed relation:** "trace" is the least-specific superproperty in the hierarchy. Requirement to architecture here is a solution-mapping relation (allocatedTo / satisfiedBy). Typing it specifically is what later permits disjointness and coverage.
- **The 2:1 is convergence, not identity:** trace 2 and trace 3 both targeting arch 2 v1 is two distinct Case 2 edges sharing an endpoint (a cardinality/coverage fact: arch 2 satisfies two requirements). Reading "same arch" as a Case 1 co-reference link would be a category error.
- **Two link kinds are painted as one:** the process-metadata *references* (req set, arch set, trace set) are Case 1 co-reference threads (they point to the same authoritative sets, not copies). They are a different kind of link from the Case 2 traces and should not share a color.
- **Assertion and provenance are bundled:** commit ids, agent/role, and tool roles are the *provenance* of the assertion (wasGeneratedBy, wasAttributedTo), not part of the relationship's meaning. Folding them into the trace id means you cannot re-establish the same relation under a new commit without losing its identity, or baseline the assertion separately from its evidence.

And the v1 quietly pins each target to a version: that is an endpoint *configuration status* (baselined versus floating), left implicit. Finally, the ASoT evaluation consuming metadata and emitting "new metadata" is really a governance activity turning *evidence* into an *authority verdict*; the output is not generic metadata, it is an authority assertion.

A cleaner representation

Same skeleton, four moves. The first two figures separate the relation layer; the third fixes what ASoT evaluation actually produces.

1. **Name the trace as a reified SemanticRelationThread, typed specifically** (allocatedTo), with explicit hasSource and hasTarget. Keeping the set is correct; addressable edges are the point.
2. **Separate the assertion from its provenance.** The trace individual holds the relation; a ProvenanceRecord (commit, agent, tool) attaches via wasGeneratedBy / wasAttributedTo. Now the same relation can be re-established under a new commit without losing identity, and can be baselined apart from its evidence.
3. **Color the two link kinds and pin the version.** Case 2 traces are indigo; the process-metadata references to the authoritative sets are Case 1 co-reference (teal). The 2:1 is annotated as coverage, not co-reference. The endpoint carries an explicit configurationStatus.
4. **Make ASoT evaluation a governance activity** that consumes provenance (evidence) and emits an authority assertion (node-role metadata). That assertion is the "new process metadata," and it becomes evidence for the next evaluation.

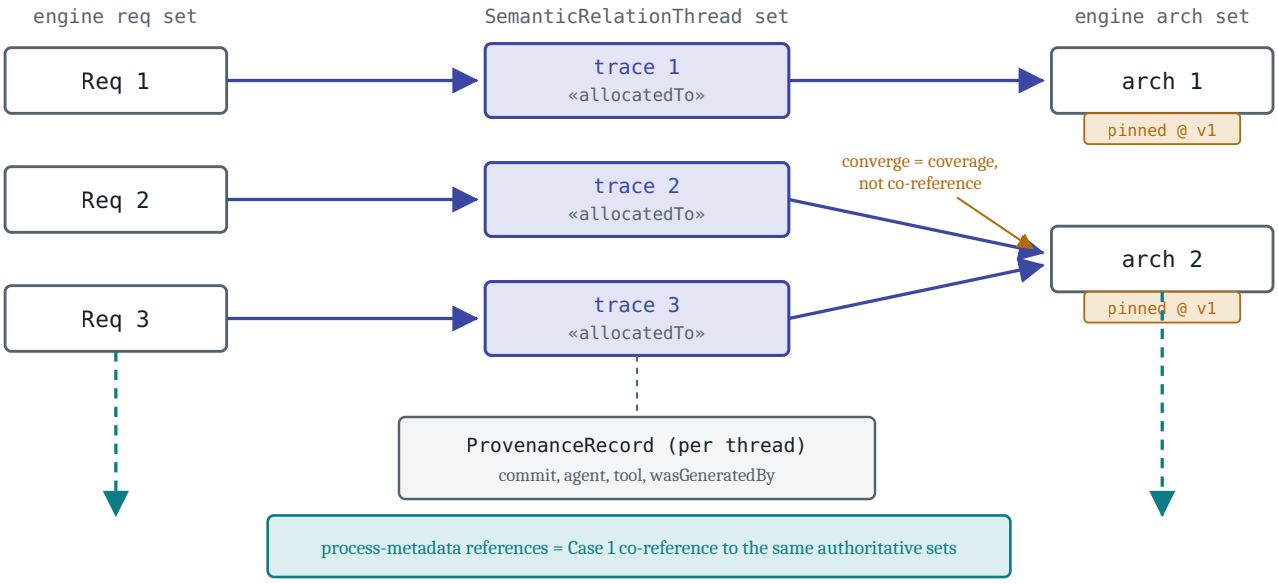

Figure 5. The trace layer re-typed: reified Case 2 threads (indigo), provenance detached from the assertion, version pinned on the endpoint, and the metadata references shown as Case 1 co-reference (teal). Convergence is labeled as coverage.

Figure 6. Authority is the verdict, provenance is the evidence, and the thread references both rather than containing either. The new process metadata is the authority assertion, which re-enters as evidence.

The conflations this untangles

- edge versus endpoints (the thread references its ends; it is not them)
- assertion versus transport/mechanism (stable claim versus swappable enforcement)
- thread versus pipeline / workflow / API (relation versus activity / orchestration / interface)
- atomic thread versus the digital thread (one edge versus the connected, traversable graph)
- Case 1 co-reference versus two Case 2 edges converging on one node (identity versus coverage)
- edge identity versus edge provenance (the assertion versus how it came to be)
- authority assertion versus provenance (the verdict versus the evidence)
- version-pinned versus floating endpoint (baseline modality on the node)

One judgment underlies all of this: treating *connectivity* as the primitive, and pipelines, workflows, and analytics as consumers of it. That is the defensible reading of the OSLC and PROV evidence, not a settled consensus; the analytical-camp definition would fold some computation into the thread itself. It is worth stating out loud when the group adopts terms, because the rest of the model (what to reify, where provenance attaches, what ASoT evaluation emits) follows from that one choice.

The sum-total process: information exchange across the digital thread

The body of this note deliberately isolates the thread from its neighbors. This addendum puts them back together and draws the larger digital-engineering information-exchange process in one place, so the distinct roles (the thread, its endpoints, the mechanism that carries the exchange, authority, and provenance) can be contrasted rather than blurred. These are working sketches for discussion, not a finished schema. Three views follow: the whole ecosystem in layers, a single exchange that sets the assertion against the mechanism carrying it, and the way an ASoT role is realized.

View 1: the layered ecosystem. Read it top to bottom as a stack of concerns. Mechanisms in the realization layer (APIs, pipelines, orchestration) actually move the information and, in doing so, generate provenance in the evidence layer. That evidence feeds the ASoT evaluation in the governance layer, which grants authority to a node. The authoritative value then propagates through Case 1 threads in the assertion layer. The thread layer is the stable backbone that survives; the mechanism layer underneath is swappable; authority sits above as a verdict, and provenance underneath as its evidence. Nothing in the mechanism layer is a thread, and nothing in the thread layer moves data by itself.

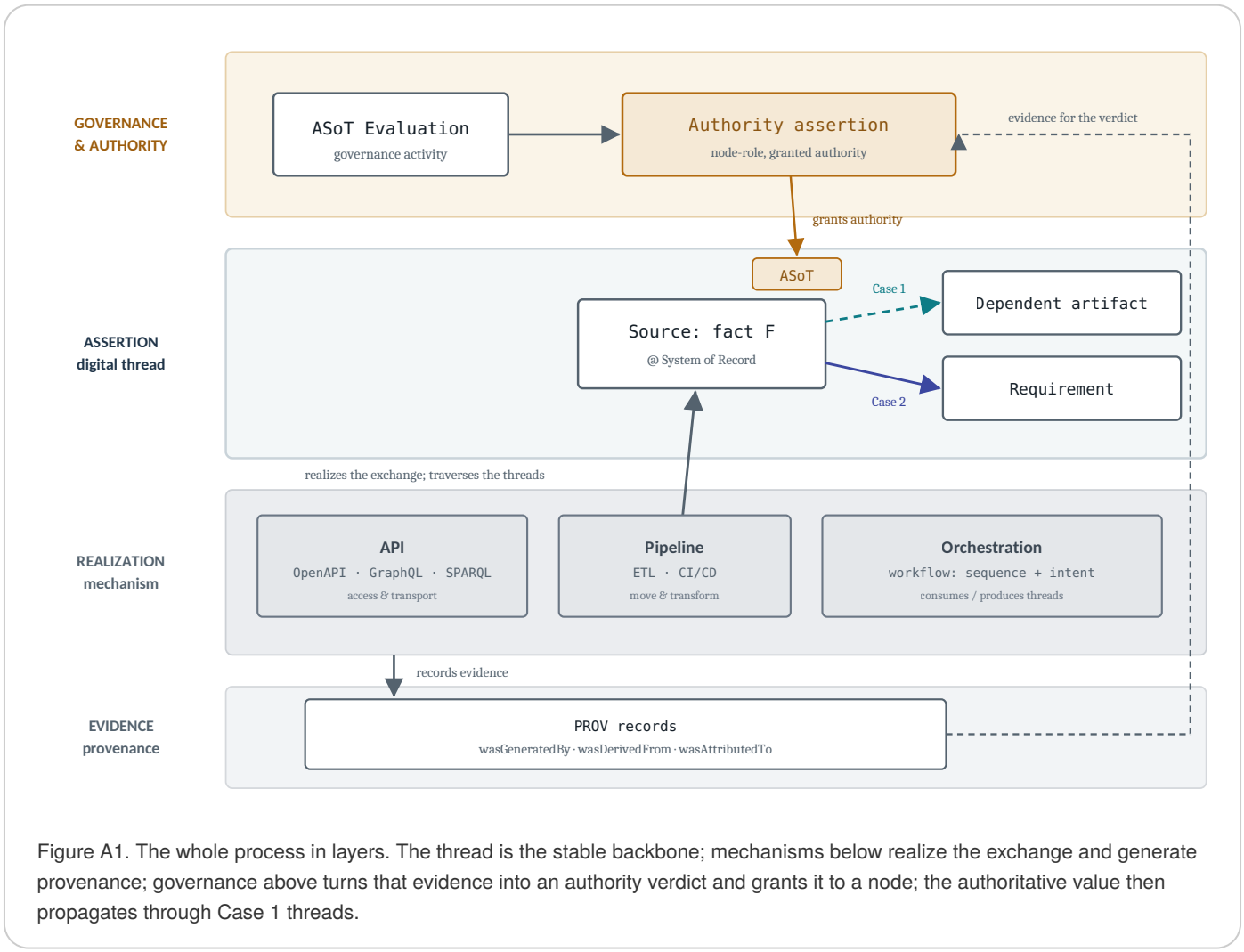

Figure A1. The whole process in layers. The thread is the stable backbone; mechanisms below realize the exchange and generate provenance; governance above turns that evidence into an authority verdict and grants it to a node; the authoritative value then propagates through Case 1 threads.

View 2: one exchange, contrasted. The same two endpoints appear in two lanes. The top lane is the thread: a Case 1 assertion that the source and target should hold the same value, which persists whether or not anything is running. The bottom lane is the exchange itself: an API reads, a pipeline transforms, an orchestration writes, and the value actually moves. The thread says the values should agree; the mechanism makes them agree. This is the sharpest contrast in the whole model: replace the entire bottom lane (swap the API, rewrite the pipeline, reorder the workflow) and the top lane is unchanged, because the assertion is not the mechanism. Authority is granted to the source from above; provenance is emitted below.

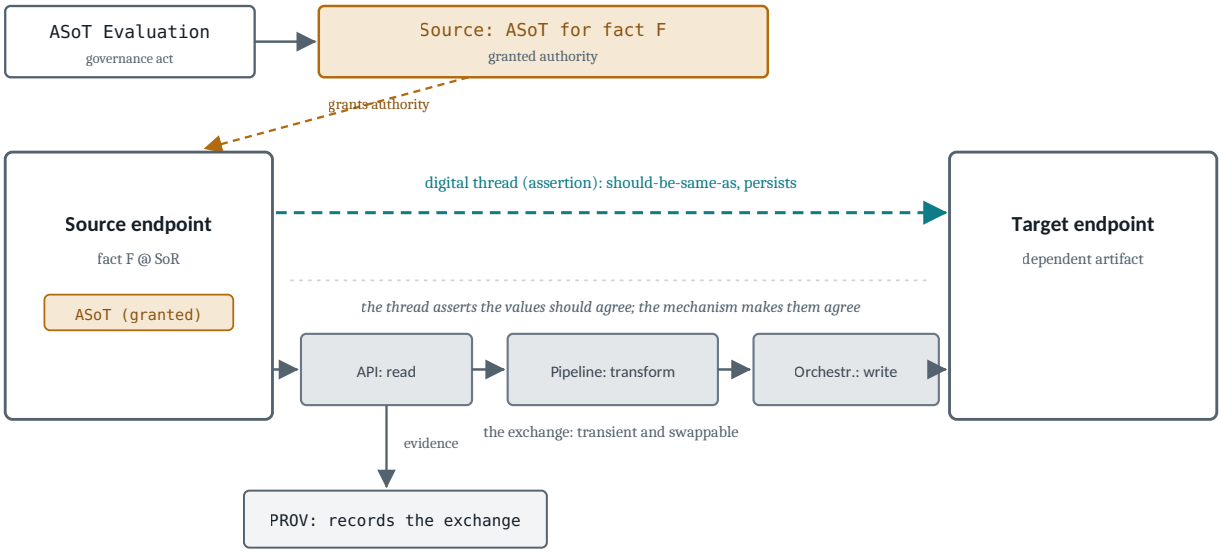

Figure A2. One exchange, two lanes, the same two endpoints. The top lane is the thread (a Case 1 assertion that persists); the bottom lane is the mechanism that carries the value (transient and swappable). Authority is granted to the source above; provenance is emitted below. Replace the whole bottom lane and the top lane is unchanged.

View 3: how the ASoT role is realized. Authority is not one thing that either exists or does not. Asserted authority (the governance metadata that a node is the ASoT for a fact) is always present once declared. Realizable authority is a disposition: the thread could produce the value on traversal but has not yet, which is exactly the virtualized or federated source (a view or query that would return the value when forced). Realized authority exists only after a query or run materializes the value and stamps it with the provenance of that traversal. The role is realized by the very process in which the authoritative value is produced and used with its granted authority. This is why baselining a virtual source is not free: to freeze it you must force realization once, then snapshot and provenance the result.

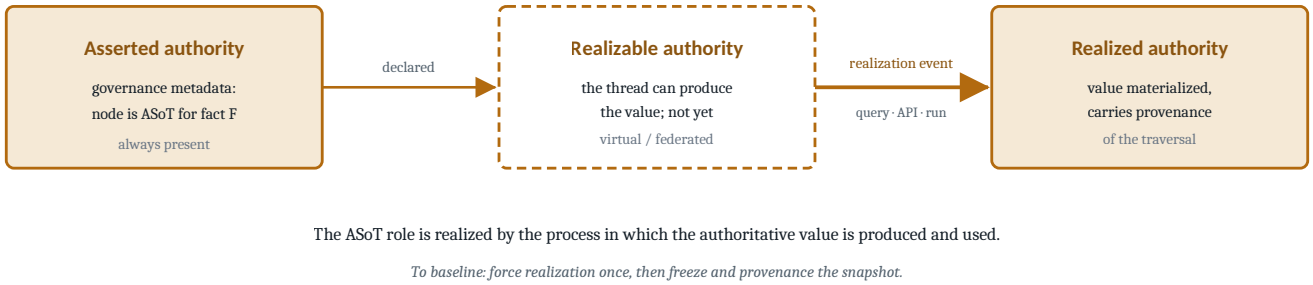

Figure A3. How an ASoT role is realized. Asserted authority (governance metadata) is always present; realizable authority is a disposition (the value would be produced on traversal, the virtualized or federated case); realized authority exists only after a query or run materializes the value and records its provenance.

The three views are the same process from different distances. View 1 is the ecosystem, View 2 is the operation of a single edge, and View 3 is the modality of the authoritative endpoint that edge depends on. For now these are flowcharts; the next step, if the group wants it, is to map each layer to who owns it in formalism (OWL for the assertion and its hierarchy, SHACL for conformance and coverage, PROV for the evidence layer, and governance rules for the authority verdict), so the picture becomes checkable rather than only illustrative.

Dr. Zachary Welz